Optional Enhancements

Automated testing or evaluation approaches

1. Automated testing can be found int the *tests* folder.
2. I also performed several manual tests and they were all succesfull:

BOOKING REFERENCE

- User asks to add luggage without providing a booking reference
  Expected outcome*:* Agent must ask for the booking reference
- User enters correct booking reference
  Expected outcome Agent validates it and returns a successful response
- User enters wrong booking reference
  Expected outcome: Agent explains the reference provided is not valid
- User enters wrong booking reference and then the correct one
  Expected outcome: Agent explains the reference provided is not valid, then validates and returns a successful response
- User changes booking reference
  Expected outcome: Agent validates the new booking reference and confirms the change of booking reference on success
- User asks if booking reference is correct
  Expected outcome: Agent asks for the booking reference, validates it, and returns a response

BAGGAGE TESTS

- User asks to add baggage providing all information
  Expected outcome: Agent asks for confirmation, providing all required details correctly: quantity, baggage name, passenger, and cost
- User asks to add baggage without providing required information
  Expected outcome: Agent asks for required information: baggage name and passenger, then once provided asks for confirmation including all details
- User asks to add baggage providing only baggage name
  Expected outcome: Agent asks for required information: passenger name, then once provided asks for confirmation including all details
- User asks to add baggage providing only passenger name
  Expected outcome: Agent asks for required information: baggage name, then once provided asks for confirmation including all details
- User asks to add baggage but cancels on confirmation request by the agent
  Expected outcome: Agent cancels the operation and confirms it to the user. The API call is not performed
- User adds baggage successfully
  Expected outcome: Agent calls the add_luggage API and returns a successful response

- User adds baggage that does not exist
  Expected outcome: Agent does not proceed and explains the baggage is not available
- User adds baggage with a passenger that does not exist
  Expected outcome: Agent does not proceed and explains the passenger is not available
- User adds baggage without quantity
  Expected outcome: Agent asks for quantity or proceeds assuming the quantity is 1
- User adds baggage with quantity
  Expected outcome: Agent manages it and the confirmation message includes the correct quantity and cost calculation
- User adds baggage with a quantity larger than available
  Expected outcome: Agent does not proceed and explains the quantity is not available
- User asks to add multiple luggage providing all information
  Expected outcome: Agent asks for confirmation, providing all required details correctly: all quantities and baggage names, passenger, and total cost
- User asks to add multiple luggage for multiple customers
  Expected outcome: Agent asks for confirmation, providing all required details correctly: all quantities and baggage names per passenger, and total cost
- User adds multiple luggage successfully
  Expected outcome: Agent calls the add_luggage API for each luggage and returns a successful response
- User adds multiple luggage for multiple passengers successfully
  Expected outcome: Agent calls the add_luggage API for each luggage with the correct passenger, and returns a successful response
- User adds multiple luggage for multiple passengers but some return an error
  Expected outcome: Agent calls the add_luggage API for each luggage and returns an error response
- User adds baggage with passenger name or baggage name slightly misspelled or partial
  Expected outcome: Agent understands correctly or asks for clarification
- User adds baggage with surname or name, but the surname or name is used by multiple customers
  Expected outcome: Agent asks for clarification
- User adds baggage with implicit terms like "add again the previous baggage"
  Expected outcome: Agent understands correctly and asks for confirmation, providing the correct baggage name
- User asks to see available baggage options
  Expected outcome: Agent shows all available luggage options per customer, including ID, name, and price
- User asks to see available baggage options after adding a baggage successfully
  Expected outcome: Agent shows updated available luggage options
- User asks to see available baggage options of a flight without available baggage slots
  Expected outcome: Agent does not proceed and returns an error response

- User changes booking reference and asks to see available baggage options
  Expected outcome: Agent provides booking options for the new booking reference
- User changes booking reference and adds baggage for the new booking reference
  Expected outcome: Agent calls the add_luggage API and adds baggage with the correct booking reference

SOMETHING GOES WRONG TESTS

- User adds baggage but API response is error "Item 1 has already been added to this booking."
  Expected outcome: Agent returns an error and sets quantity to 0 for the luggage
- User adds multiple bags but some return an error
  Expected outcome: Agent returns an error explaining what happened
- User adds baggage with a booking reference of a flight without available baggage slots
  Expected outcome: Agent returns an error and does not proceed
- User adds baggage that is not available
  Expected outcome: Agent returns an error and does not proceed
- User adds baggage but there is a Baggage mismatch error
  Expected outcome: Expected to show error and ask for human takeover
- Service failure: OpenAI
  Expected outcome: Agent communicates service is unavailable and triggers human escalation
- Service failure: API
  Expected outcome: Agent communicates service is unavailable and triggers human escalation
- Service failure: Booking reference lookup
  Expected outcome: Agent communicates service is unavailable and starts human escalation request
- Service failure: Get luggage options
  Expected outcome: Agent communicates service is unavailable and starts human escalation request
- Service failure: Add luggage
  Expected outcome: Agent communicates service is unavailable and starts human escalation request

HUMAN TAKEOVER TESTS

- User tries to add a wrong booking reference 3+ times
  Expected outcome: Agent communicates the reference is wrong and asks to enter it again; after the 3rd attempt, it starts a human escalation request
- User asks to remove baggage or changes mind
  Expected outcome: Agent explains it cannot remove baggage and starts a human escalation request
- User asks for human takeover explicitly
  Expected outcome: Agent asks for confirmation

- User asks for human takeover explicitly without providing a booking reference
  Expected outcome: Agent asks for confirmation and a booking reference, and then proceeds
- User asks for human takeover explicitly immediately and provides both confirmation and booking reference at the same time
  Expected outcome: Agent asks for confirmation and then proceeds using the booking reference provided
- User asks for human takeover explicitly but it is already triggered
  Expected outcome: Agent communicates a human will contact them already and does not call the API / escalation
- Human takeover escalation API call includes full conversation transcript
  Expected outcome: Check it

MISC

- User asks unrelated questions
  Expected outcome: Agent does not answer the question and explains it can only answer questions related to flights, baggage, and escalations
- User greets and thanks agent
  Expected outcome: Agent replies correctly

Observability and monitoring considerations

1. Added agent log feature to print agent tool calling

Guardrails and safety mechanisms

1. Added a guardrail to prevent the agent from replying to questions unrelated to luggage, flights, passengers, or human escalation. This guardrail has been implemented as a system prompt.

Ideas for how the solution could evolve into a broader booking management assistant

1. Add option to remove luggage: Implement functionality allowing users to remove baggage from their booking.
2. Sync bookings for logged-in users: Automatically sync the logged-in user's status so they do not need to manually enter a booking reference.
3. Add message delays: Implement a slight delay before sending user message to agent to allow users to send multiple consecutive messages before the agent responds.